

同步與非同步

- **15-1: 同步與非同步的差別**
 同步與非同步的差別.....15-2
- **15-2: async、await、Task**
 Task.....15-4
 async.....15-5
 await.....15-6
- **15-3: EF Core 實際範例適合使用非同步的情況**
 EF Core同步寫法.....15-7
 EF Core非同步寫法.....15-8
 EF Core的async方法.....15-9
- **15-4: 適合使用非同步的情況**
 適合使用非同步的情況.....15-10

Module 15

同步與非同步的差別-1

- 同步：
 - 「事情沒做完，就站在原地等」。
 - 服務生把餐點送進廚房後，站在廚房門口等待，不能服務其他客人。
- 非同步：
 - 「事情處理中，先把執行緒讓給其他工作，完成後再回來繼續」。
 - 非同步的重點不是讓單一查詢變快，也不是自動建立新執行緒，而是避免資料庫、網路、檔案等 I/O 等待期間占住執行緒，讓 Web 伺服器能同時處理更多請求。
 - 服務生送單後先服務其他客人，餐點完成時再回來送餐。

同步與非同步的差別-2

- 同步：會阻塞執行緒

```
public IActionResult SyncDemo()
{
    Thread.Sleep(3000);

    return Content("同步作業完成");
}
```

Thread.Sleep(3000) 的 3 秒內，執行緒只能等待。

// Thread.Sleep(3000) 的 3 秒內，執行緒只能等待。

- 非同步：等待時釋放執行緒

```
public async Task<IActionResult> AsyncDemo()
{
    await Task.Delay(3000);

    return Content("非同步作業完成");
}
```

await Task.Delay(3000) 執行時：

1. 方法執行到 await 。
2. 發現工作還沒完成。
3. 暫停目前方法。
4. 執行緒回到執行緒集區，可以處理其他請求。
5. 3 秒後，找一個可用執行緒繼續執行後面的程式。

Task.Delay() 適合教學模擬等待，正式系統通常是等待資料庫、API 或檔案操作。

Task

- 代表「目前可能還沒完成的工作」。

```
public void SendEmail()
{
    // 寄信程式碼
}

0 個參考
public async Task SendEmailAsync()
{
    // 寄信程式碼
}
```

```
public Customer GetCustomer()
{
    return new Customer();
}

0 個參考
public async Task<Customer> GetCustomerAsync()
{
    return new Customer();
}
```

- 可以把 **Task<Customer>** 想成取餐號碼牌：現在還沒有餐點，但未來完成時會拿到一份**Customer**。

async

- **async**定義非同步的方法。

```
public async Task<Customer> GetCustomerAsync()  
{  
    return new Customer();  
}
```

- 單純加上 **async** 不會讓方法自動在背景執行，也不會自動建立執行緒。方法仍會同步執行到下一個尚未完成的 **await**。

await

- **await**呼叫非同步的方法。

```
await SendEmailAsync();
```

- **await**等待非同步工作完成，但不阻塞目前執行緒。

```
Customer customer = await GetCustomerAsync();
```

- 這會把 **Task<Customer>** 裡面的結果取出來，因此變數型別是 **Customer**，不是 **Task< Customer >**。

EF Core 同步寫法

```
public IActionResult Details(int id)
{
    Product? product = _context.Products
        .FirstOrDefault(p => p.ProductId == id);

    if (product == null)
    {
        return NotFound();
    }

    return View(product);
}
```

- **FirstOrDefault()** 查詢資料庫時，執行緒會一直等待。

EF Core 非同步寫法

```
public async Task<IActionResult> Details(int id, CancellationToken cancellationToken)
{
    Product? product = await _context.Products
        .AsNoTracking()
        .FirstOrDefaultAsync(p => p.ProductId == id, cancellationToken);

    if (product == null)
    {
        return NotFound();
    }

    return View(product);
}
```

- **FirstOrDefault()** 查詢資料庫時，執行緒會一直等待。

EF Core的async方法

- 通常是「真正執行 SQL」的方法才有非同步版本：

同步	非同步
<code>ToList()</code>	<code>ToListAsync()</code>
<code>FirstOrDefault()</code>	<code>FirstOrDefaultAsync()</code>
<code>SingleOrDefault()</code>	<code>SingleOrDefaultAsync()</code>
<code>Count()</code>	<code>CountAsync()</code>
<code>Any()</code>	<code>AnyAsync()</code>
<code>SaveChanges()</code>	<code>SaveChangesAsync()</code>

- LINQ語法：Where()、OrderBy()、Select()、Include()沒有async。**

適合使用非同步的情況

- 適合用於等待外部資源的 I/O 操作：
 - EF Core 資料庫查詢
 - 呼叫 Web API
 - HttpClient
 - 上傳、下載或讀寫檔案
 - Stream 讀寫
 - 寄送郵件
 - 等待計時器